

Lab0: GitLab

复旦大学
黄果 23301030064

一、实验步骤与结果

1. 完成 Git 基本配置

在 WSL 终端中首先检查 Git 是否安装成功，并配置提交记录中使用的用户名与邮箱。

```
1. git --version
2. git config --global user.name "<username>"
3. git config --global user.email "<useremail>"
4. git config --global --list

squaredog@DESKTOP-92P7DI3:/mnt/c/Users/SquareDoggg$ git --version
git version 2.43.0

squaredog@DESKTOP-92P7DI3:/mnt/c/Users/SquareDoggg$ git config --global --list
user.name=SquareDoggg
user.email=SquareDoggggg@gmail.com
```

图 1 Git 版本与用户信息配置结果

2. 配置 GitHub SSH 公钥

由于实验主要在 WSL 环境中完成，SSH 密钥应在 WSL 中生成并使用。生成密钥后，将 .pub 公钥内容添加到 GitHub 的 SSH and GPG keys 页面，并使用 ssh -T 验证认证是否成功。私钥必须保存在本机，不能上传或公开。

```
1. ssh-keygen -t ed25519 -C "<useremail>"
2. cat ~/.ssh/id_ed25519.pub
3. ssh -T git@github.com

squaredog@DESKTOP-92P7DI3:/mnt/c/Users/SquareDoggg$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDu2CyGqRpP9Q7SvQE4ltmLGKWr7JHrtQVva7/hb1jFn SquareDoggggg@gmail.com
squaredog@DESKTOP-92P7DI3:/mnt/c/Users/SquareDoggg$ ssh -T git@github.com
Hi SquareDoggg! You've successfully authenticated, but GitHub does not provide shell access.
```

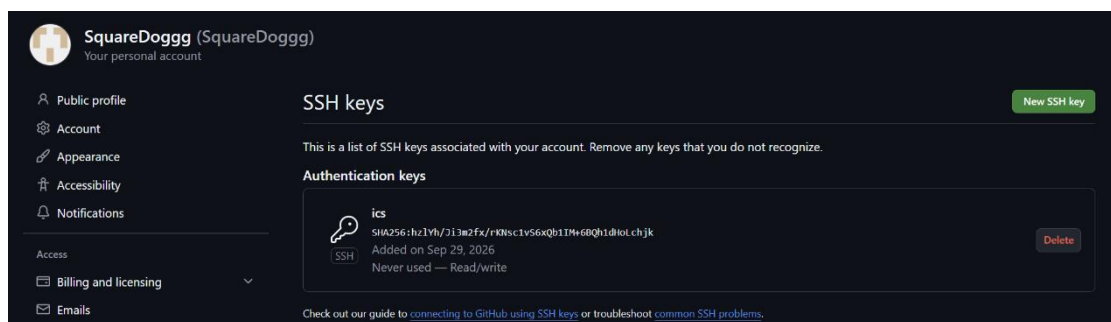


图 2 GitHub SSH 认证成功

3. 使用模板仓库建立个人仓库并克隆

在课程提供的模板仓库中选择 Use this template→Create a new repository，建立个人实验仓库。随后复制 SSH 地址，在 WSL 中执行 git clone，并进入仓库目录。

```
1. git clone <remote URL>
2. cd <repository>
3. git status
```

```
squaredog@DESKTOP-92P7DI3:/mnt/c/Users/SquareDoggg$ cd ~
squaredog@DESKTOP-92P7DI3:~$ git clone git@github.com:SquareDoggg/Lab0.git
Cloning into 'Lab0'...
remote: Enumerating objects: 28, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 28 (delta 4), reused 20 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (28/28), 47.29 KiB | 44.00 KiB/s, done.
Resolving deltas: 100% (4/4), done.
```

图 3 个人仓库建立并成功克隆到本地

4. 完成 main.c 的 TODO 并进行首次提交

根据实验要求修改 main.c 中的 TODO。完成修改后先用 git status 检查状态，再将修改加入暂存区并提交。提交信息应尽量说明本次修改的目的，而不是只写 update 或 change。

```
1. git status
2. git add main.c
3. git commit -m "Complete TODO"
```

```
int main(){
    //TODO: only enable tests THAT YOU CAN PASS
    //test1(1);
    //test2(NULL, 0);
    //test3(0);
    //test4(0);
    //test5();
    //test6(0);
    //test7(0);
    //test8(0);
    //test_progress_bar(0);

    printf("Hello World!\n");
    {
        try{
        }
        catch{
            printf("test FAILED on main\n");
        }
    }
    assert(__check_err_stk());
    return 0;
}
```

```

squaredog@DESKTOP-92P7DI3:~/Lab0$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
squaredog@DESKTOP-92P7DI3:~/Lab0$ cd coroutine/
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ vim main.c
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git add .
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git commit -m "Complete TODO"
[main 69c6670] Complete TODO
 1 file changed, 1 insertion(+), 1 deletion(-)
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 357 bytes | 357.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:SquareDoggg/Lab0.git
 e56c296..69c6670  main -> main

```

图 4 main.c 修改及首次 commit 结果

5. 创建 feature 分支制造合并冲突

为了保证合并时确实发生冲突，需要让 main 和 feature 两个分支分别修改 main.c 的同一行或同一小段内容，并各自提交。例如，先从 main 创建 feature 分支，在 feature 中修改一行字符串并提交；再切回 main，把同一行改成不同内容并提交。

1. git switch -c feature
2. # 修改 main.c
3. git add main.c
4. git commit -m "Modify message on feature branch"
5. git switch main
6. # 再次修改 main.c 的同一位置，但改成不同内容
7. git add main.c
8. git commit -m "Modify message on main branch"

```

squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git switch -c feature
Switched to a new branch 'feature'
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ vim main.c
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git add main.c
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git commit -m "Modify message on feature branch"
[feature 8893098] Modify message on feature branch
 1 file changed, 1 insertion(+), 1 deletion(-)
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ vim main.c
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git add main.c
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git commit -m "Modify message on main branch"
[main 35a4a68] Modify message on main branch
 1 file changed, 1 insertion(+), 1 deletion(-)

```

```
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git log --graph --oneline --all
* 35a4a68 (HEAD -> main) Modify message on main branch
| * 8893098 (feature) Modify message on feature branch
|/
* 69c6670 (origin/main, origin/HEAD) Complete TODO
```

图 5 feature 与 main 分支分别完成提交

6. 合并 feature 分支并处理冲突

在 main 分支执行 git merge feature。由于两个分支修改了 main.c 的同一位置，Git 无法自动决定保留哪一份内容，因此报告 CONFLICT。此时打开 main.c，可以看到 <<<<<<<、=====、>>>>>>> 三组冲突标记。根据最终希望保留的逻辑手动修改文件，删除冲突标记，再执行 git add 和 git commit 完成合并。

1. git switch main
2. git merge feature
3. # 手动编辑 main.c，删除冲突标记并保留最终代码
4. git add main.c
5. git commit -m "merge: resolve conflict between main and feature"
6. git status

```
int main(){
    //TODO: only enable tests THAT YOU CAN PASS
    //test1(1);
    //test2(NULL, 0);
    //test3(0);
    //test4(0);
    //test5();
    //test6(0);
    //test7(0);
    //test8(0);
    //test_progress_bar(0);

<<<<<< HEAD
    printf("Hello:)World!\n");
=====
    printf("HelloWorld!\n");
>>>>>> feature
    {
        try{
        }
        catch{
            printf("test FAILED on main\n");
        }
    }
    assert(__check_err_stk());
    return 0;
}
```

```
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git merge feature
Auto-merging coroutine/main.c
CONFLICT (content): Merge conflict in coroutine/main.c
Automatic merge failed; fix conflicts and then commit the result.
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ vim main.c
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git add main.c
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git commit -m "Resolve conflict between main and feature"
[main de3ae4b] Resolve conflict between main and feature
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
```

图 6 执行 git merge feature 后出现合并冲突

7. 检查提交历史并推送到 GitHub

冲突解决后，使用图形化提交历史或 `git log` 检查分支与合并关系。确认代码、实验报告均已加入 `main` 分支后，将最终结果推送到 GitHub。

1. `git log --graph --oneline --decorate --all`
2. `git add .`
3. `git commit -m "Hello World! "`
4. `git push`

```
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git log --graph --oneline --decorate --all
*   de3ae4b (HEAD -> main) Resolve conflict between main and feature
| \
|  * 8893098 (feature) Modify message on feature branch
| * | 35a4a68 Modify message on main branch
|/
```

```
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git add .
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git commit -m "Hello World!"
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
squaredog@DESKTOP-92P7DI3:~/Lab0/coroutine$ git push
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 20 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 1.03 KiB | 1.03 MiB/s, done.
Total 12 (delta 10), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (10/10), completed with 3 local objects.
To github.com:SquareDoggg/Lab0.git
   69c6670..de3ae4b  main -> main
```

图 7 Git 最终提交

二、思考题

1. 你之前有过多人协同开发的经历吗？如果有，使用什么方式分工协作？

没有多人协同开发经历，因此向几位有相关经验的朋友进行了了解。

计算机学院的一位同学曾与朋友合作开发游戏 Mod，他们会在开发前先共同确定双方都认可的 `main` 版本，之后非必要不再频繁修改主分支，并提前划分任务以尽量减少重复开发；如果某一部分需要等待他人完成，则先设置占位内容，并及时补充说明，后期还会单独建立分支，只要功能完成且不会导致程序崩溃即可合并。某 AI4S 实习生的协作方式更接近模块化分工，即由带教明确分配某项具体功能，完成后通过接口交付，由带教集成到整体项

目中。某无人驾驶行业从业者则更强调沟通和文档的重要性，认为多人协作中应通过完善说明文档和频繁交流及时同步进展。综合来看，多人开发的核心并不只是把任务分开，还需要明确主线版本、减少功能重叠、合理使用分支、预先约定接口，并通过文档和持续沟通降低协作成本。

2. Git 为什么设计“暂存—提交”两个步骤？

暂存区的核心作用是把当前工作区所有变化与这一次真正想提交的变化分开。开发过程中可能同时修改多个文件，甚至在同一文件中完成多项工作；如果每次只能把全部修改直接提交，就很难保持提交的原子性。通过 `git add`，开发者可以主动选择本次提交的内容，在 `commit` 前再次检查，从而减少把调试代码、临时文件或无关改动误提交的风险。最终形成的小而清晰的提交也更方便代码审查、问题定位和版本回退。

3. `git branch` 与 `git branch -a` 的区别

`git branch` 默认只列出本地分支，并用*标记当前所在分支；`git branch -a` 中的 `-a` 表示 `all`，会同时显示本地分支和远程跟踪分支（通常形如 `remotes/origin/main`）。因此，前者适合查看当前本地开发分支，后者适合整体确认本地与远程已有的分支情况。另外，`git branch -r` 只显示远程跟踪分支。

4. 为什么要学习 Git？结合三篇阅读材料谈谈你的理解。

《Commit Message 和 Change Log 编写指南》主要讨论如何通过规范化的提交信息提高项目历史的可读性。文章介绍了 Header、Body 和 Footer 等基本结构，其中 Header 常采用 `type(scope):subject` 的形式，并通过 `feat`、`fix`、`docs`、`refactor` 等类型区分不同性质的修改。规范化的 Commit Message 不仅能够帮助开发者快速了解一次提交的目的，还便于筛选提交记录、定位问题，并可以进一步用于自动生成 Change Log。因此，提交信息本身也是版本管理的一部分，而不仅仅是附加说明。

《Gitflow 使用规范》主要从分支管理的角度介绍团队开发流程。Git Flow 通常将 `main/master` 作为稳定发布分支，将 `develop` 作为日常开发分支，并根据开发任务建立 `feature`、`release` 和 `hotfix` 等临时分支。不同分支承担不同职责，并规定从哪个分支创建、最终合并到哪个分支。其核心作用是将多人协作中的开发流程结构化，使新功能开发、测试、发布和紧急修复相互隔离，减少多人直接修改主分支所带来的冲突和误操作风险。

《语义化版本 2.0.0》则从软件发布的角度说明如何使用版本号表达软件变化。语义化版本采用 X.Y.Z 的形式，其中主版本号表示存在不兼容的 API 修改，次版本号表示增加向下兼容的新功能，修订号表示进行了向下兼容的问题修复。此外，规范还定义了先行版本号和构建元数据，并强调已经发布的版本不应直接修改，而应通过发布新版本表达变化。这样，版本号不仅是编号，也能够向软件使用者和依赖方传递兼容性信息。

通过这三篇材料，我认为学习 Git 的意义并不只是掌握 `git add`、`git commit` 和 `git push` 等命令，而是学习如何系统地管理软件开发过程中不断发生的变化。Commit Message 解决的是“每次修改如何被清楚记录”的问题，Git Flow 解决的是“多人和多个任务如何并行高效协作”的问题，语义化版本解决的是“完成的修改如何组织成正式版本并向外传递变化”的问题。三者分别对应提交、分支和版本三个层次，共同构成较完整的版本管理流程。Git 能够保留项目历史、支持回退和分支开发，也能够形成可分析的开发记录；相关研究表明，Git 仓库中的 commit 历史可以用于分析开发者之间的协作关系^[5]，版本控制活动数据也可以用于软件开发工作量估计^[6]。因此，对日常学习和后续项目开发而言，掌握 Git 不仅能够降低代码修改和协作中的风险，也有助于形成更规范、可追踪的开发习惯。

三、实验总结

本次 Lab0 从最基本的 Git 配置和提交操作开始，进一步完成了分支创建、分支切换、主动制造 merge conflict、手动解决冲突以及向 GitHub 推送最终结果。通过本次实验，理解了 Git 中“提交”和“分支”的含义：commit 是一次可追踪的项目状态记录，branch 则是组织并行开发历史的轻量机制。

三篇材料说明了规范化版本控制包含多个层次：一次提交需要清晰的 Commit Message；多人开发需要合理的分支工作流；正式发布需要明确的版本语义。后续实验中需要在修改代码前先确认当前分支，在提交前检查 `git status` 与 `diff`，并尽量保持一次提交只完成一个明确目的，以降低调试、回退和协作成本。

四、建议

建议后续实验继续保留提交树作为辅助工具，并在实验说明中加入错误提交和错误合并的恢复示例，说明如何撤销误操作或恢复文件。相比只展示正常流程，这类场景更能帮助初学者建立对版本状态变化的直观认识。

五、参考文献

- [1] 复旦大学计算机系统基础课程组. Lab0: GitLab[EB/OL]. 2026-09-14[2026-09-30]. <https://ics-26fall-fdu.github.io/>.
- [2] 阮一峰. Commit message 和 Change log 编写指南[EB/OL]. 2016-01-06[2026-09-30]. https://www.ruanyifeng.com/blog/2016/01/commit_message_change_log.html.
- [3] dafay. Gitflow 使用规范 [EB/OL]. 2024-03-18[2026-09-30]. <https://www.dafaycoding.com/article/git-gif-flow>.
- [4] Preston-Werner T. 语义化版本 2.0.0[EB/OL]. [2026-09-30]. <https://semver.org/lang/zh-CN/spec/v2.0.0.html>.
- [5] Gote C, Scholtes I, Schweitzer F. Analysing Time-Stamped Co-Editing Networks in Software Development Teams using git2net[J]. Empirical Software Engineering, 2021, 26: 75.
- [6] Robles G, Capiluppi A, Gonzalez-Barahona J M, Lundell B, Gamalielsson J. Development effort estimation in free/open source software from activity in version control systems[J]. Empirical Software Engineering, 2022, 27(6): 135.